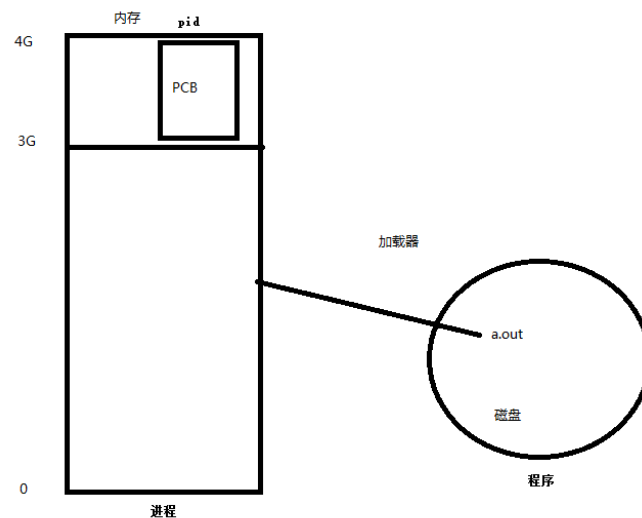


进程概念

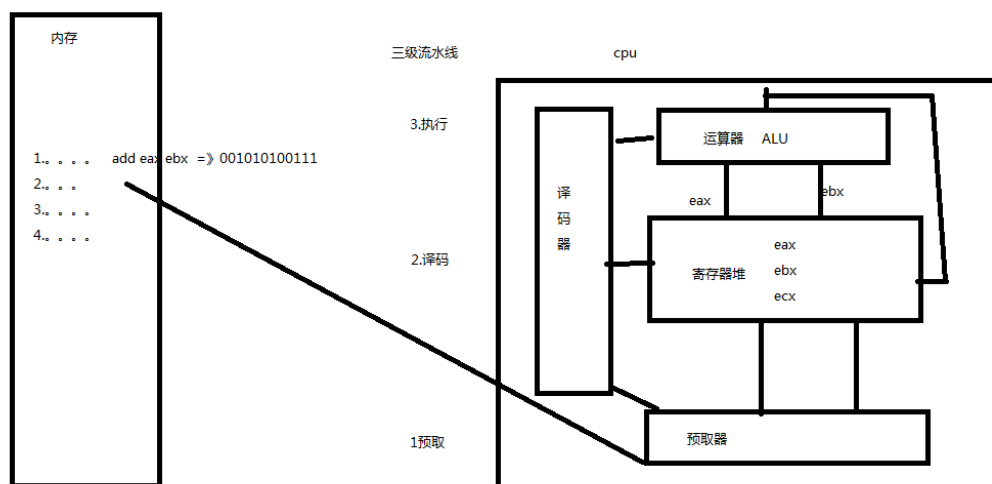
1、进程：运行的程序就是进程，每个进程都有进程控制块PCB，Linux内核的进程控制块是task_struct结构体。

每一个进程ID号。路径：/usr/src/linux-h*****/include/linux/sched.h

系统每加载一个进程时，都会虚拟一个0到4G的内存空间来存储信息。PCB存放在内存3G到4G空间，也称为内核空间。0到3G为用户空间。每个进程的用户空间都是不一样的，而内核空间都是共用物理内存空间。

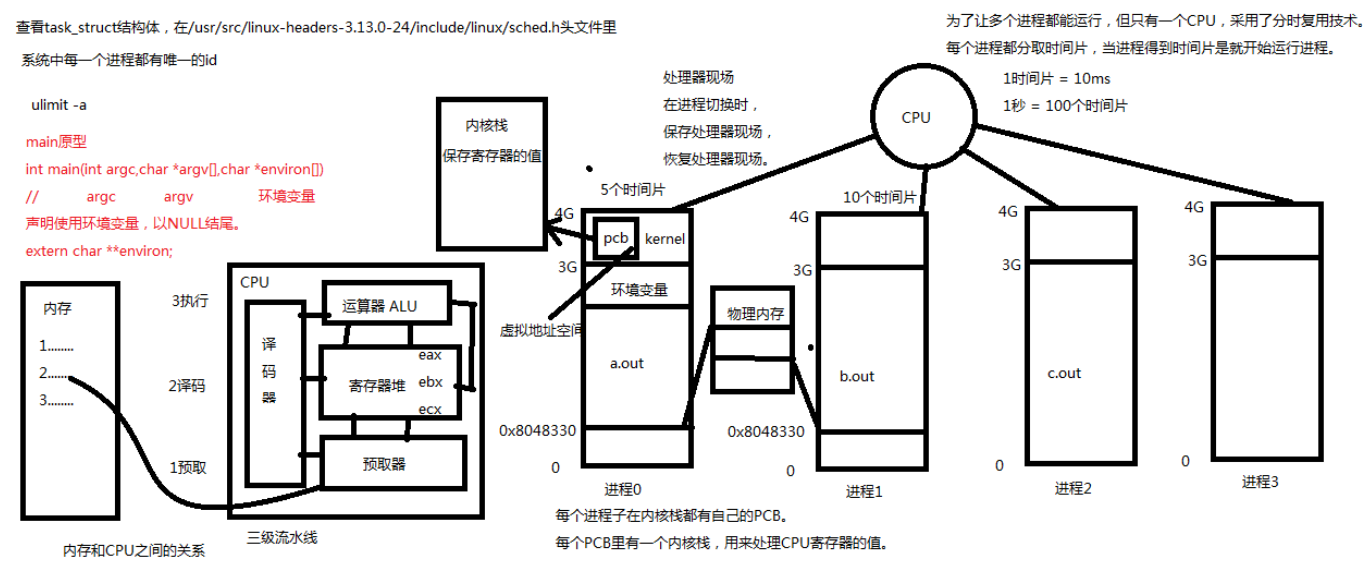


2、CPU三级流水线：预取，译码，执行。



3、分时复用：任务采用时间片轮，分别执行多任务。

在宏观上是同步，在微观上是异步。处理器现场，在进程切换时，保存处理器现场和恢复处理器现场。内核空间有一个内核栈，用来保存寄存器的值，每个PCB里又都维护了一个内核的栈，负责处理器现场。PCB保存着虚拟内存地址的映射表。进程的环境变量在内核空间下的用户空间中。



4、ulimit -a 进程资源的上限值（Resource Limit）

当前工作目录：Current Working Directory

umask 掩码

文件描述符表，包含很多指向file结构体的指针。

5、进程环境变量

按照惯例，环境变量字符串都是name=value键值对形式，大多数name由大写字母加下划线组成，一般把name的部分叫做环境变量，value的部分则是环境变量的值，以NULL结尾。

获取环境变量：

```
1 #include <stdio.h>
2 //int main(int argc, char *argv[], char *environ[])//main函数的默认形式
3
4 int main(int argc, char *argv[])
5 {
6     extern char **environ;
7     int i = 0;
8 }
```

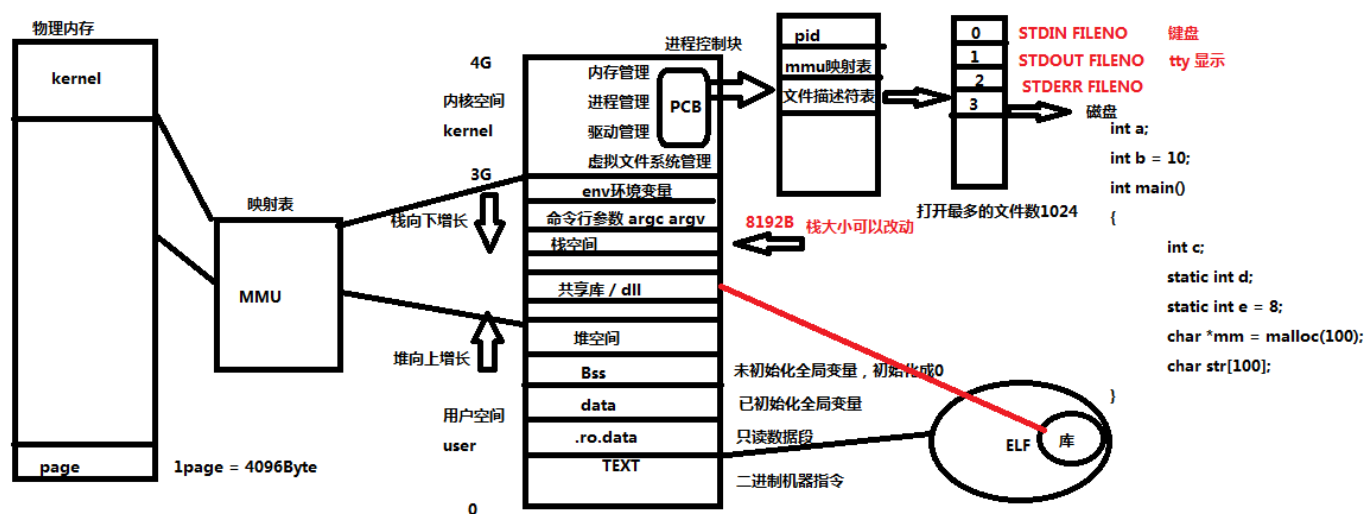
```

9   while(envIRON[i] != NULL)
10      printf("%s\n",envIRON[i++]);
11
12   return 0;
13 }

```

如果不传入环境变量时，那么就使用系统默认的环境变量。

用envIRON指针可以查看所有环境变量字符串，但是不够方便，如果给出name要在环境变量表中查找它对应的value，可以用getenv函数。



6、getenv函数（使用name，获取环境变量的值）

```

1 #include <stdlib.h>
2
3 char *getenv(const char *name);

```

getenv的返回值是指向value的指针，若未找到则为NULL。

7、setenv函数（修改环境变量）

```

1 #include <stdlib.h>

```

```

2
3 int setenv(const char *name, const char *value, int rewrite); //设置修改
4
5 void unsetenv(const char *name); //删除

```

getenv和**setenv**函数若成功则返回为0，若出错则返回非0。

setenv将环境变量**name**的值设置为**value**。如果已存在环境变量**name**，那么

若**rewrite**非0，则覆盖原来的定义；1

若**rewrite**为0，则不覆盖原来的定义，也不返回错误。

unsetenv删除**name**的定义。即使**name**没有定义也不返回错误。

```

xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ abc=123
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ echo $abc
123
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ unset abc
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ echo $abc
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ 

```

临时环境变量的设置与删除

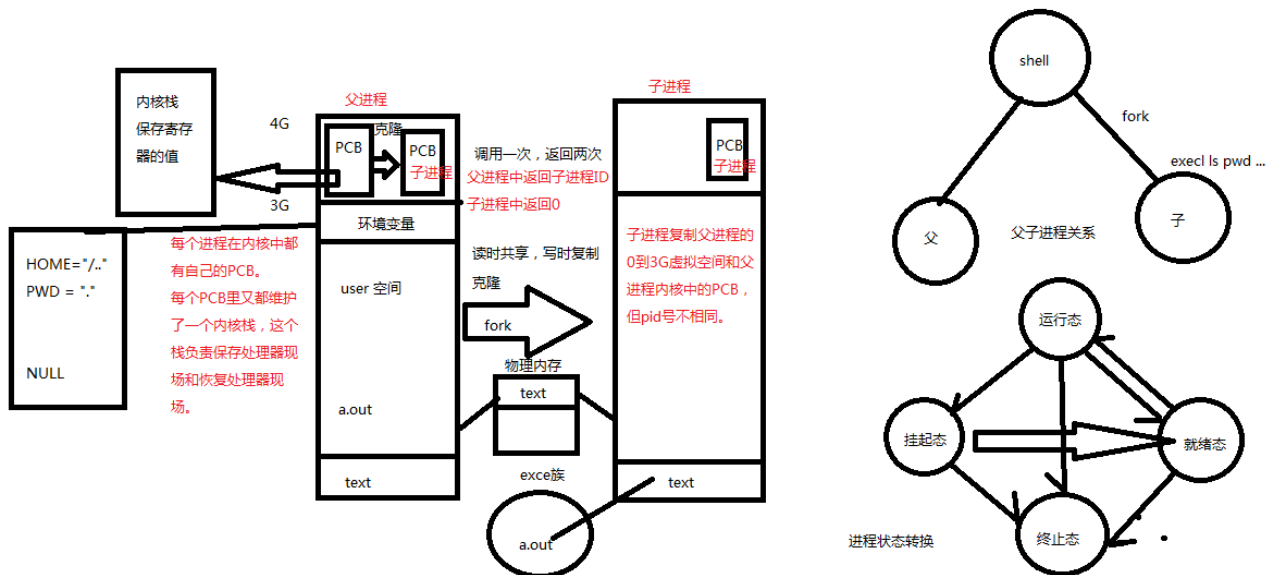
除

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main(void)
6 {
7     char buf[4096];
8     printf("%s\n",getenv("PATH"));
9     //返回的字符串地址慎用，使用字符数组来处理，避免数据丢失
10    char *str = getenv("PATH");//获取环境变量
11    getcwd(buf,sizeof(buf));//获取路径
12
13    strcat(buf,":");
14    strcat(buf,str);
15
16    setenv("PATH",buf,1);//修改环境变量，1 表示覆盖
17    printf("%s\n",getenv("PATH"));
18
19    return 0;
20 }

```

8、进程状态：就绪态，运行态，挂起态，终止态。



修改进程资源限制，软限制可改，最大值不能超过硬限制，硬限制只有root用户可以修改。

```
1 #include <sys/time.h>
2
3 #include <sys/resource.h>
4
5 int getrlimit(int resource, struct rlimit *rlim); //获取
6
7 int setrlimit(int resource, const struct rlimit *rlim); //设置
```

```
54 #@student - maxlogins 4
55 * soft nfile 65536
56 * hard nfile 100000
57
58 # End of file
/etc/security/limits.conf
```

查看进程资源限制

cat /proc/self/limits

ulimit -a

9、进程原语fork函数

```
1 #include <unistd.h>
2
3 pid_t fork(void); //创建一个子进程
```

子进程复制父进程的0到3G空间和父进程内核的PCB，但ID号不同。

fork调用一次返回两次。

- 1、父进程中返回子进程ID;
- 2、子进程中返回0;
- 3、读时共享，写时复制。

Fork()

{

- 1、创建子进程地址空间
- 2、克隆父进程PCB
- 3、克隆父进程用户空间0~3G
- 4、子进程独立出来
- 5、返回子进程ID/或0。

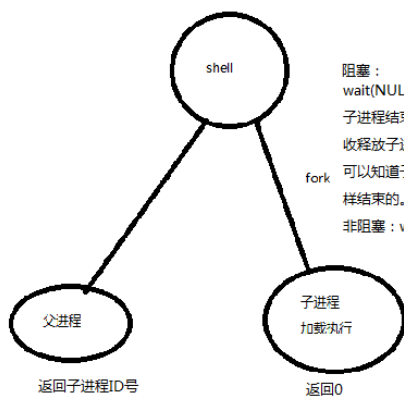
}

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     pid_t pid;
8     int i = 0; //读时共享，写时复制
9
10    pid = fork(); //调用一次，返回两次。
11
12    if(pid > 0) //父进程
```

```

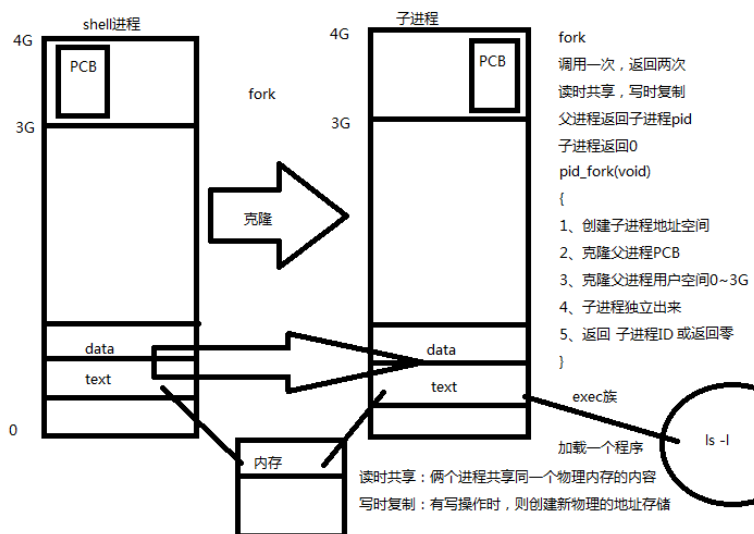
13 {
14     while(1)
15     {
16         printf("I am parent %d\n",i+=3);
17         sleep(1);
18     }
19 }
20 else if(pid == 0)//子进程
21 {
22     while(1)
23     {
24         printf("I am child %d\n",i++);
25         sleep(1);
26     }
27 }
28 else //小于0 出错
29 {
30     perror("fork");
31     exit(1);
32 }
33
34 return 0;
35 }

```



僵尸进程：子进程结束了，用于父进程对子进程死后验尸。没有释放子进程PCB，释放了0~3G。

孤儿进程：父进程结束了，子进程还在运行。将子进程托付给1号进程。父进程负责回收释放PCB。



```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>

```

```

4 //生成100个子进程
5 int main(void)
6 {
7     pid_t pid;
8     int i;
9     for(i = 0;i < 100; ++i)
10    {
11        pid = fork();
12        if(pid == 0)
13            break;
14        else if(pid < 0)
15        {
16            perror("fail");
17            exit(-1);
18        }
19    }
20    if(pid == 0)
21    {
22        printf("child pid = %d,ppid = %d\n",getpid(),getppid());
23    }
24    if(pid > 0)
25    {
26        printf("parent pid = %d,ppid = %d\n",getpid(),getppid());
27        while(1);
28    }
29    return 0;
30 }

```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(int argc,char *argv[])
6 {
7     pid_t pid;
8
9     if(argc < 2)
10    {
11        printf("./a.out filename\n");
12        exit(1);

```



```

13     }
14     int num = atoi(argv[1]);
15
16     while(num--)
17     {
18         pid = fork();
19         if(pid == 0)
20         {
21             break;
22         }
23         if(pid < 0)
24         {
25             perror("fork fail");
26             exit(-1);
27         }
28     }
29     if(pid == 0)
30     {
31         printf("ppid = %d, childpid = %d\n",getppid(),getpid());
32     }
33     if(pid > 0)
34     {
35         printf("ppid = %d, parentpid = %d\n",getppid(),getpid());
36         while(1);
37     }
38
39     return 0;
40 }

```

10、进程的相关函数

1、getpid/getppid函数

```

1 #include <sys/types.h>
2 #include <unistd.h>
3
4 pid_t getpid(void); //返回调用进程的PID号
5
6 pid_t getppid(void); //返回调用进程父进程的PID号

```

2、getgid函数

```
1 #include <unistd.h>
2 #include <sys/types.h>
3
4 gid_t getgid(void); //返回实际用户组ID
5
6 gid_t getegid(void); //返回有效用户组ID
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     pid_t pid;
8     int i = 0;
9
10    pid = fork();
11
12    if(pid > 0){
13        while(1){
14            printf("%d I am parent %d,my parent is %d,child is %d\n",i+=3,getpid(),getppid(),getgid());
15            sleep(1);
16        }
17    }
18    else if(pid == 0){
19        while(1){
20            printf("%d I am child %d,my parent is %d\n",i++,getpid(),getppid());
21            sleep(1);
22        }
23    }else{
24        perror("fork");
25        exit(1);
26    }
27
28    return 0;
29 }
```

11、exec族函数

当子进程调用exec族函数时，该进程的用户空间代码和数据完全被新程序替换，从新程序的启动例程开始执行。调用exec并不创建新进程，所以调用exec前后该进程的id并未改变。

```
1 #include <unistd.h>
2
3 int execl(const char *path, const char *arg, ...);
4 int execlp(const char *file, const char *arg, ...); //在PATH环境变量查找
5 int execlenv(const char *path, const char *arg, ..., char *const envp[]); //可以修改环境变量
6
7 int execv(const char *path, char *const argv[]);
8 int execvp(const char *file, char *const argv[]);
9 int execve(const char *path, char *const argv[], char *const envp[]); //系统调用
```

如果调用成功则加载新的程序从启动代码开始执行，不再返回，如果调用出错则返回-1，所以exec函数只有出错的返回值而没有成功的返回值。

l 命令行参数列表

p 搜索file时使用path变量

v 使用命令行参数数组

e 使用环境变量数组，不使用进程原有的环境变量，设置新加载程序运行的环境变量

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     printf("%d\n", getpid());
7     // 路径 //argv[0] 占位
8     //execl("/bin/ls", "/bin/ls", "-l", NULL);
9     //execl("/bin/ls", "yzmin", "-l", NULL);
10    execl("b.out", "b.out", "hello", "123", NULL);
11    printf("world\n");
12
13    return 0;
14 }
```

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5     int i = 0;
6     printf("%d\n", getpid());
7     while(argv[i] != NULL)
8     {
9         printf("argv[%d] = %s\n", i, argv[i]);
10        i++;
11    }
12    return 0;
13
14 }
```

```
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ ./a.out
36956
36956
argv[0] = b.out
argv[1] = hello
```

```
argv[2] = 123
```

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     printf("hello\n");
7     //    路径        //argv[0]占位
8     //execl("/bin/ls","/bin/ls","-l",NULL);
9     execl("/bin/ls","yzmin","-l",NULL);
10    printf("world\n");
11
12    return 0;
13 }
```

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     printf("hello\n");
7     //execl("/bin/ls","/bin/ls","-l",NULL);//可变参数
8     //execl("b.out","b.out","123","jjjj",NULL);
9     //execlp("ls","ls","-l",NULL);//默认使用环境变量
10    //execl("/bin/ls","xwp","-l",NULL);
11
12    char *argvv[] = {"ls","-l",NULL};
13    execv("/bin/ls",argvv);//指针数组
14    printf("%d\n",getpid());
15
16    //execl("b.out","b.out","hello","123",NULL);
17    printf("world\n");
18
19    return 0;
20 }
```

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     pid_t pid;
8     pid = fork();
9
10    if(pid > 0)
11    {
12        while(1)
13        {
14            printf("parent\n");
15            sleep(1);
16        }
17    }
18    else if(pid == 0)
19    {
20        execl("b.out", "b.out", "yzmin", "256", NULL); //b.out 替换了子进程原先的代码段
21        while(1)
22        {
23            printf("child\n");
24            sleep(1);
25        }
26    }
27    else
28    {
29        perror("fork");
30        exit(-1);
31    }
32
33    return 0;
34 }

```

1、upper.c

```

1 #include <stdio.h>
2

```

```

3 int main(void)
4 {
5     int ch;
6     while((ch = getchar()) != EOF)
7     {
8         putchar(toupper(ch));
9     }
10
11     return 0;
12 }

```

2、 wrapper.c

```

1 #include <unistd.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4 #include <fcntl.h>
5
6 int main(int argc, char *argv[])
7 {
8     int fd;
9     if(argc != 2)
10     {
11         fputs("usage: Wrapper file\n", stderr);
12         exit(1);
13     }
14
15     fd = open(argv[1], O_RDONLY);
16     if(fd < 0)
17     {
18         perror("open");
19         exit(1);
20     }
21
22     dup2(fd, STDIN_FILENO);
23     close(fd);
24     execl("./upper", "upper", NULL);
25     perror("exec ./upper");
26     exit(1);
27 }

```

将文件作为输入转换为大写输出。

12、wait/waitpid函数

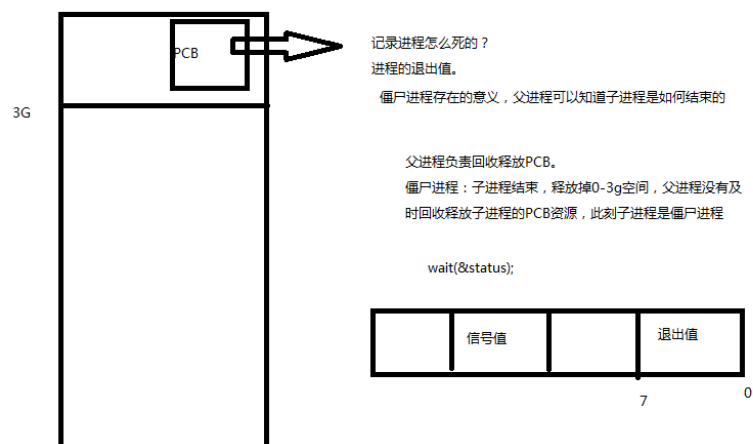
僵尸进程：子进程退出，父进程没有回收子进程资源（PCB），则子进程变成僵尸进程。

孤儿进程：父进程先于子进程结束，则子进程成为孤儿进程，子进程的父进程成为1号进程init进程，称为init进程领养孤儿进程。

```
1 #include <sys/types.h>
2 #include <sys/wait.h>
3
4 //阻塞函数，如果没有子进程结束，就等待
5 pid_t wait(int *status); //status为传出参数，0至7位表示退出值，其他表示信号值。
6
7 //非阻塞函数
8 pid_t waitpid(pid_t pid, int *status, int options);
9
10 参数pid值：
11 < -1 回收指定进程组内的任意子进程
12  -1 回收任意子进程（子）
13  0 回收和当前调用waitpid一个组的所有子进程
14 > 0 回收指定ID的子进程
```

子进程结束：当父进程没有回收子进程退出的资源，那么资源还是存储在PCB中，等待父进程去回收资源。父进程负责回收释放PCB。僵尸进程，是为了父进程知道子进程怎么死的，子进程的0~3G空间已经释放。

echo \$? 查看返回的值



1、wait.c

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 #include <stdlib.h>
5
6 int main(void)
7 {
8     pid_t pid;
9     int i = 0;
10
11     while(i < 1000)
12     {
13         pid = fork();
14         if(pid == 0)
15         {
16             break;
17         }
18         else if(pid < 0)
19         {
20             perror("fork");
21             exit(-1);
22         }
23         i++;
24     }
25
26     if(pid == 0)
27     {
28         int n;
29         srand(time(NULL));
30         n = rand()%10;
31         while(n--)
32         {
33             printf("I am child %d\n",getpid());
34             sleep(1);
35         }
36     }
```



```

36     }
37     else if(pid > 0)
38     {
39         int wpid;
40         while(1)
41         {
42             wpid = wait(NULL); //如果没有子进程结束，wait阻塞等待。如果没有子进程，则出错。
43             printf("wait for %d\n", wpid);
44         }
45     }
46
47     return 0;
48 }

```

2、wait2.c

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/wait.h>
4  #include <stdlib.h>
5
6  int main(void)
7  {
8      pid_t pid;
9      int status;
10     pid = fork();
11
12     if(pid > 0)
13     {
14         printf("yzmin\n");
15         //wait函数等待子进程结束，然后回收。
16         printf("kill child %d\n", wait(&status)); //获取status退出值
17
18         if(WIFEXITED(status)) //判断status是否正常退出
19             printf("exit status %d\n", WEXITSTATUS(status));
20
21         if(WIFSIGNALED(status)) //判断status是否信号触发退出
22             printf("exit status %d\n", WTERMSIG(status));
23
24         printf("world\n");
25     }
26     else if(pid == 0)

```

```

26     case 1 (pid == 0)
27     {
28         while(1)
29         {
30             printf("hello\n");
31             sleep(1);
32             //return 2;
33         }
34     }
35     else
36     {
37         perror("fork");
38         exit(1);
39     }
40
41     return 0;
42 }

```

Kill -9 -gpids 杀掉一个进程组。

1、waitpid.c

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <stdlib.h>
4  #include <errno.h>
5  #include <sys/wait.h>
6
7  void sys_err(char *str)
8  {
9      perror(str);
10     exit(1);
11 }
12
13 int main(int argc, char *argv[])
14 {
15     pid_t pid;
16     int n = atoi(argv[1]);
17     while(n--)
18     {
19         if((pid = fork()) == 0)
20         {

```

```

21         break;
22     }
23     else if(pid < 0)
24     {
25         sys_err("fork");
26     }
27 }
28
29 if(pid == 0)
30 {
31     printf("I am %d,start %s ...\n",getpid(),argv[2]);
32     execvp(argv[2],argv+2);
33 }
34
35 if(pid > 0)
36 {
37     int wpid,i = 0;
38     //WNOHANG 期望不要被挂起，没有子进程结束，则立即返回
39     //waitpid 返回值 -1: 出错，没有子进程 0: 有子进程，但是没有终止，非阻塞
40     while((wpid = waitpid(-1,NULL,WNOHANG)) >= 0)
41     {
42         if(wpid == 0)
43         {
44             printf("wait for child %d\n",i++);
45         }
46         else
47         {
48             printf("child %d is over\n",wpid);
49         }
50         sleep(1);
51     }
52
53     if((wpid < 0) && (errno == ECHILD))
54     {
55         printf("all childs is over,bye\n");
56     }
57 }
58
59 return 0;
60 }

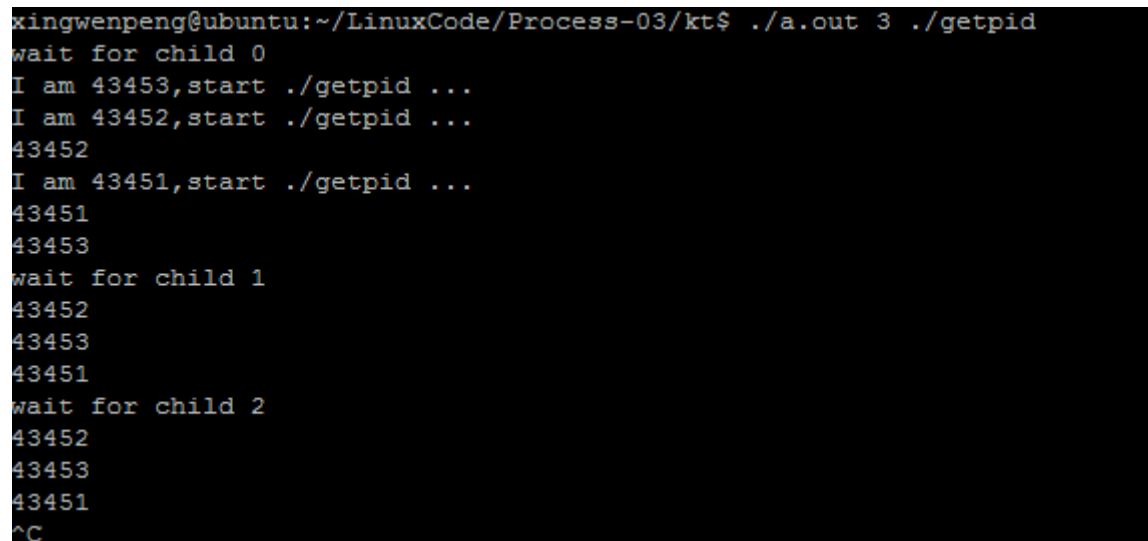
```

2、waitpid2.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main(void)
6 {
7     pid_t pid;
8     int status;
9
10    pid = fork();
11    if(pid > 0)
12    {
13        printf("yzmin\n");
14        printf("kill child %d\n",waitpid(-1,&status,WNOHANG));  
//非阻塞
15        if(WIFEXITED(status))
16            printf("exit status %d\n",WEXITSTATUS(status));
17
18        if(WIFSIGNALED(status))
19            printf("exit status %d\n",WTERMSIG(status));
20
21        printf("world\n");
22    }
23    else if(pid == 0)  
//孤儿进程
24    {
25        while(1)
26        {
27            printf("hello\n");
28            sleep(1);
29        }
30    }
31    else
32    {
33        perror("fork");
34        exit(1);
35    }
36    return 0;
37 }
```

3、getpid.c

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(void)
5 {
6     while(1)
7     {
8         printf("%d\n",getpid());
9         sleep(1);
10    }
11
12    return 0;
13 }
```



```
xingwenpeng@ubuntu:~/LinuxCode/Process-03/kt$ ./a.out 3 ./getpid
wait for child 0
I am 43453,start ./getpid ...
I am 43452,start ./getpid ...
43452
I am 43451,start ./getpid ...
43451
43453
wait for child 1
43452
43453
43451
wait for child 2
43452
43453
43451
^C
```

waitpid函数非阻塞，让父进程与子进程同时处理不同的任务。

13、多进程拷贝

```
1 #include <stdio.h>
2 #include <sys/stat.h>
3 #include <sys/types.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6 #include <stdlib.h>
7 #include <string.h>
```

```

8 #include <sys/mman.h>
9
10 #define N 5
11
12 struct child_task {
13     char src[256];
14     char dest[256];
15     int addr;
16     int size;
17     int num;
18 };
19
20 void sys_err(const char *str)
21 {
22     perror(str);
23     exit(-1);
24 }
25
26 int main(int argc, char *argv[])
27 {
28     int nfork, i, task_addr, task_size, total_size, fddest, tmpfd;
29     pid_t pid;
30     struct stat sbuf;
31     int *mem;
32
33     if (argc < 3) {
34         printf("./mycp src dest\n");
35         exit(-1);
36     }
37     if (argc == 3)
38         nfork = N;
39     else
40         nfork = atoi(argv[3]);
41
42     if (stat(argv[1], &sbuf) < 0)
43         sys_err("stat");
44     total_size = sbuf.st_size;
45     fddest = open(argv[2], O_CREAT|O_RDWR, 0644);
46     if (fddest < 0)
47         sys_err("open");
48     if (lseek(fddest, total_size-1, SEEK_SET) < 0)
49         sys_err("lseek");
50     write(fddest, "\0", 1);
51     close(fddest);

```

```

52 //struct child_task work[nfork];
53 struct child_task *work = malloc(nfork * sizeof(struct child_task));
54 task_size = total_size / nfork;
55 for (i = 0; i < nfork-1; i++) {
56     strcpy(work[i].src ,argv[1]);
57     strcpy(work[i].dest ,argv[2]);
58     work[i].addr = i * task_size;
59     work[i].size = task_size;
60     work[i].num = i;
61 }
62 /* 最后一个进程拷贝字节 */
63 strcpy(work[i].src ,argv[1]);
64 strcpy(work[i].dest ,argv[2]);
65 work[i].addr = i * task_size;
66 work[i].size = total_size - work[i].addr;
67 work[i].num = i;
68
69 tmpfd = open("tmpfile", O_CREAT|O_RDWR|O_EXCL, 0664);
70 if (tmpfd < 0)
71     sys_err("open tmpfile");
72
73 ftruncate(tmpfd, nfork*sizeof(int));
74
75 mem = mmap(NULL, nfork*sizeof(int), PROT_READ|PROT_WRITE, MAP_SHARED, tmpfd, 0);
76 if (mem == MAP_FAILED)
77     sys_err("mmap");
78 memset(mem, 0, nfork*sizeof(int));
79 //int mem[5]; mem[0]=20 mem[1] = 50 ...
80
81 unlink("tmpfile"); //删除临时文件
82
83 for (i = 0; i < nfork; i++) {
84     if ((pid = fork()) < 0)
85         sys_err("fork");
86     else if (pid == 0) {
87         /*in child*/
88         int fdsrc, fddest, j, len;
89         char buf[4096];
90         fdsrc = open(work[i].src, O_RDONLY);
91         if (fdsrc < 0)
92             sys_err("open src");
93         fddest = open(work[i].dest, O_WRONLY);
94         if (fddest < 0)

```

```

95         sys_err("open dest");
96         if (lseek(fdsrc, work[i].addr, SEEK_SET) < 0)
97             sys_err("lseek fdsrc");
98         if (lseek(fddest, work[i].addr, SEEK_SET) < 0)
99             sys_err("lseek fddest");
100
101         for (j = 0; j < work[i].size; j++) {
102             len = read(fdsrc, buf, 1);
103             if (len < 0)
104                 sys_err("read");
105             write(fddest, buf, len);
106             mem[work[i].num]++;
107             //                usleep(100000);
108         }
109         close(fdsrc);
110         close(fddest);
111         break;
112     }
113     if(pid > 0)
114     {
115         printf("parent %d\n",i);
116     }
117 }
118
119 if (pid > 0) {
120     pid = fork();
121     /* 打印拷贝进程 */
122     if (pid == 0) {
123         int i, sum = 0;
124         while (sum < total_size) {
125             sum = 0;
126             for (i = 0; i < nfork; i++) {
127                 sum += mem[i];
128             }
129             printf("%d / %d\n", sum, total_size);
130             usleep(100000);
131         }
132     }
133     else if (pid > 0) {
134         for (i = 0; i < nfork+1; i++)
135             wait(NULL);
136     }
137     else {
138         sys_err("fork");

```



```
139     }
140 }
141
142 free(work);
143 return 0;
144 }
```